TransGuard

Memoria técnica del proyecto de escritorio

Sistema de apoyo a la operación de una terminal portuaria

Python · Tkinter · MongoDB · Ollama (llama3)

Contenido

1. Ficha del proyecto 2

2. Puesta en marcha 2

2.1 Requisitos . . . 2

2.2 Ejecución . . . 2

2.3 Variables de entorno opcionales . . . 2

3. Diseño interno 3

Archivos . . . 3

4. Datos que guarda la terminal 3

5. Motor de decisiones (lógica proposicional) 4

5.1 Premisas del dominio portuario . . . 4

5.2 Reglas . . . 4

5.3 Tablas de verdad de las reglas nuevas . . . 4

5.4 Trazabilidad y análisis . . . 4

6. Clasificador mixto de incidentes 5

6.1 Categorías del dominio . . . 5

6.2 Flujo . . . 5

6.3 Prompt del clasificador (texto íntegro) . . . 5

6.4 Experimento sobre el dataset (32 correos etiquetados a mano) . . . 5

7. Asistente consultivo (RAG) 6

Prompt del asistente (texto íntegro) . . . 6

8. Registro de riesgos éticos 6

9. Pantalla de escritorio (ejercicio 2) 7

System prompt del tutor (texto íntegro) . . . 8

10. Cómo se comprobó 8

11. Limitaciones conocidas 8

12. Solución rápida de problemas 8

1. Ficha del proyecto

Sistema de apoyo a la operación de una **terminal portuaria** desarrollado en Python + Tkinter. Aplica un LLM local (Ollama) a dos productos: un tutor académico de Inteligencia Artificial y un panel de control de acceso de unidades con clasificación de incidentes, asistente consultivo y registro de riesgos éticos.

Dato	Valor
Nombre del sistema	TransGuard
Dominio	Terminal portuaria (unidades TRK, muelles, básculas, TOS)
Interfaz	Escritorio — tkinter + ttkbootstrap (tema oscuro)
Persistencia	MongoDB (`transguard`) con respaldo automático a JSON local
LLM	Ollama, modelo `llama3` por defecto
Idioma	Español
Ejercicios	1: tutor de IA · 2: panel integral TransGuard

2. Puesta en marcha

2.1 Requisitos

- Python 3.12+
- MongoDB en `mongodb://localhost:27017` (o Atlas vía variable de entorno)
- Ollama activo con `llama3` (`ollama pull llama3`)
- Paquetes: `ollama pymongo pydantic ttkbootstrap pillow fpdf2`

2.2 Ejecución

```
:: Tutor de IA (ejercicio 1)
cd Ulises\ejercicio1_tutor_ia
python tutor_ia.py

:: TransGuard (ejercicio 2)
cd Ulises\ejercicio2_transguard
python run.py
```

2.3 Variables de entorno opcionales

Variable	Uso	Valor por defecto
`MONGO_URI`	Cadena de conexión	`mongodb://localhost:27017`
`MONGO_DB`	Base de datos	`transguard`
`OLLAMA_MODEL`	Modelo del clasificador/asistente	`llama3`

Sin MongoDB el sistema arranca igual en **modo local**: persiste en `datos/local_fallback.json` y lo anuncia en la barra inferior y en la pestaña Configuración. Sin Ollama el clasificador trabaja solo por reglas y el asistente muestra los registros recuperados sin redacción del modelo.

3. Diseño interno

El ejercicio 2 se separa en tres capas sin dependencia circular:

```
ui/                  ← pantallas (pestañas tkinter, widgets canvas)
   │   usa
nucleo/              ← reglas, clasificador, asistente, reportes, seed
   │   usa
nucleo/db.py         ← Store: MongoDB o JSON local (misma API)
```

`ui/` nunca habla directo con pymongo ni con Ollama: todo pasa por `nucleo`. Las llamadas lentas (LLM, Mongo) corren en hilos vía `run_bg()` para no congelar la ventana.

Archivos

```
Ulises/
├─ ejercicio1_tutor_ia/
│  └─ tutor_ia.py            chat + resumen de historial
└─ ejercicio2_transguard/
   ├─ run.py                 arranque
   ├─ config.py              parámetros y rutas
   ├─ nucleo/
   │  ├─ db.py               Store/Coleccion + agregación por semana
   │  ├─ reglas.py           motor proposicional y tablas de verdad
   │  ├─ clasificador.py     reglas + LLM + fusión híbrida
   │  ├─ esquemas.py         validación pydantic
   │  ├─ asistente.py        RAG: búsqueda → contexto → respuesta citada
   │  ├─ llm.py              cliente Ollama con latencia medida
   │  ├─ reportes.py         CSV / JSON / PDF
   │  └─ seed.py             datos de demostración
   ├─ ui/                   app_tk + 10 pestañas + widgets
   ├─ datos/correos_etiquetados.json   32 correos etiquetados a mano
   └─ tests/test_nucleo.py   29 pruebas unitarias
```

4. Datos que guarda la terminal

MongoDB `transguard`, 8 colecciones:

Colección	Contenido
`camiones`	Padrón: `camion_id` (TRK-###), `placa` (ABC-123-D), `empresa` (naviera), `autorizacion`, `certificacion_vigente`, `certificacion_dias_restantes`
`accesos`	Bitácora de ingresos: premisas `P Q R S D`, salidas `A E Z Y`, `decision`, `semaforo`, `explicacion` (paso a paso), `operador`, `timestamp`
`incidentes`	`remitente`, `asunto`, `cuerpo`, `clasificacion` (categoría/prioridad/fuente/detalle), `datos_extraidos`, `estado` (nuevo / en_atencion / cerrado), `requiere_revision_humana`, `historial[]`
`riesgos_eticos`	`modulo`, `descripcion`, `categoria`, `probabilidad`, `impacto`, `mitigacion`, `residuales`, `historico[]`
`evaluaciones_llm`	`run_id`, `modo`, `modelo`, `respuesta` (exactitudes + matriz), `latencia`, `total_correos`
`conversaciones`	Chat del asistente: `pregunta`, `respuesta`, `fuentes[]`, `modo`
`configuracion`	Modelo elegido, umbral de confianza, modo simulación, vigilante de turno
`meta`	Sellos de siembra de datos demo

Agregación (panel de control, “Incidentes por semana”): `$group` por `clasificacion.categoria` + `$isoWeek`/`$isoWeekYear` sobre `timestamp`. En modo local la misma agrupación se calcula con `datetime.isocalendar()`.

5. Motor de decisiones (lógica proposicional)

5.1 Premisas del dominio portuario

Premisa	Significado
P	Unidad con autorización previa de ingreso
Q	El peso rebasa el límite de la terminal
R	Mercancía catalogada como peligrosa
S	Operador con licencia/certificado vigente
D	Documentación portuaria completa (BL, manifiesto)

5.2 Reglas

```
A = P ∧ S ∧ ¬Q     ingreso libre
E = P ∧ (R ∨ Q)    canal de inspección
Z = P ∧ ¬D         retención documental        (nueva)
Y = P ∧ Q ∧ R      protocolo de emergencia     (nueva)
```

Justificación de las reglas nuevas

- `Z`: una unidad autorizada que llega sin el BL o el manifiesto no puede desconsolidar; se retiene en el andén de revisión documental hasta regularizar, aunque las demás reglas la dejaran pasar.
- `Y`: sobrepeso y mercancía peligrosa juntos multiplican el riesgo (falla estructural + posible derrame); bloquea el acceso hasta permiso especial.

Precedencia: `Y > Z > E > A`. Si `P` es falso no hay vía de ingreso: `denegado`.

5.3 Tablas de verdad de las reglas nuevas

`Z = P∧¬D` (Q, R, S fijas en falso):

P	D	Z
V	V	F
V	F	**V**
F	V	F
F	F	F

`Y = P∧Q∧R` se cumple en **1** de las 8 combinaciones de (P, Q, R): solo cuando las tres son verdaderas.

5.4 Trazabilidad y análisis

Cada evaluación guarda en `accesos` la explicación paso a paso (`P=V (…)`, `A = P∧S∧¬Q = V∧V∧V = V`, regla que disparó la decisión final).

Análisis exhaustivo de las 32 combinaciones (pestaña Simulador):

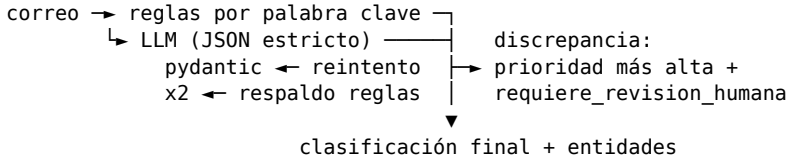
- Solapamientos: ZʌE en 6 casos, YʌE en 4, ZʌA en 2, ZʌY en 2 — todos resueltos por la precedencia.
- Redundantes: **ninguna** (ningún par de reglas coincide en las 32 salidas).
- Contradicciones irresolubles: **ninguna**.

6. Clasificador mixto de incidentes

6.1 Categorías del dominio

`carga_peligrosa` · `peso_excedido` · `acceso_indebido` · `falla_equipo` · `falla_sistema` · `estado_conductor` · `otro`

6.2 Flujo



Entidades extraídas: `placa` (ABC-123-D), `camion_id` (TRK-###), `peso_reportado_kg`, `ubicacion` (muelle/andén/grúa/dársena…).

6.3 Prompt del clasificador (texto íntegro)

```
Eres el clasificador de incidentes de la terminal portuaria TransGuard.
Analiza el correo y contesta EXCLUSIVAMENTE con un objeto JSON válido (sin
markdown ni texto extra) con esta estructura exacta:

{
  "categoria": "<carga_peligrosa | peso_excedido | acceso_indebido |
                falla_equipo | falla_sistema | estado_conductor | otro>",
  "prioridad": "<baja | media | alta | critica>",
  "entidades": {
    "placa": "<placa tipo ABC-123-D o null>",
    "camion_id": "<id tipo TRK-### o null>",
    "peso_reportado_kg": <numero en kg o null>,
    "ubicacion": "<lugar citado o null>"
  },
  "resumen": "<frase corta del incidente>"
}

Guía:
- carga_peligrosa: derrames, fugas, químicos, inflamables, gas, solventes.
- peso_excedido: báscula por encima del límite, sobrecarga.
- acceso_indebido: personas o unidades sin autorización, vallas forzadas.
- falla_equipo: grúas, cámaras, lectores, básculas dañadas o apagadas.
- falla_sistema: TOS, aplicaciones, pantallas, errores de software.
- estado_conductor: fatiga, somnolencia, conductor en mal estado.
- prioridad critica: peligro inmediato; alta: seguridad comprometida;
  media: operación afectada; baja: lo demás.
- Copia solo datos presentes en el correo; si falta un dato usa null.
```

6.4 Experimento sobre el dataset (32 correos etiquetados a mano)

Ejecutado desde la pestaña Evaluación, modo **reglas**:

Métrica	Resultado
Exactitud en categoría	**90.6 %** (29/32)

Métrica	Resultado
Exactitud en prioridad	75 % (24/32)
Latencia media	< 1 ms (sin LLM)

Matriz de confusión (esperada → obtenida):

```
carga_peligrosa → carga_peligrosa   4      peso_excedido  → peso_excedido  5
acceso_indebido → acceso_indebido   4      falla_equipo   → falla_equipo   5
falla_sistema   → falla_sistema     5      estado_conductor → estado_conductor 4
otro            → otro              2
estado_conductor → otro   1   (correo 24: "aliento alcohólico")
carga_peligrosa  → otro   1   (correo 25: "kemiko/corosibo" con faltas)
acceso_indebido  → otro   1   (correo 27: redacción muy informal)
```

Hallazgo ético real: los 3 errores de categoría corresponden a correos con ortografía informal o vocabulario no previsto — evidencia directa del riesgo de sesgo documentado en la matriz de riesgos. El modo LLM e híbrido se evalúan desde la misma pestaña (llama3 tarda ~90 s por correo en CPU) y cada corrida queda guardada en `evaluaciones_llm` para comparar.

7. Asistente consultivo (RAG)

El chat de la pestaña Asistente nunca contesta de memoria:

1. Extrae referencias de la pregunta (TRK-###, placas ABC-123-D, palabras como *incidente / acceso / riesgo*).
2. Consulta `camiones`, `accesos`, `incidentes` o `riesgos_eticos` según la intención.
3. Construye un contexto de máximo 8 registros reales.
4. El LLM responde solo con ese contexto, citando `[coleccion#id]`.
5. Sin registros → respuesta fija: *“No tengo información registrada sobre eso.”*

Prompt del asistente (texto íntegro)

```
Eres el asistente de la terminal portuaria TransGuard.
Contesta la pregunta del operador usando UNICAMENTE la información del
CONTEXTO (registros reales de la base de datos).

Reglas:
1. Cita el registro de origen con su etiqueta, p. ej. [accesos#abc123].
2. Si el contexto NO contiene la respuesta, responde exactamente:
   "No tengo información registrada sobre eso." No inventes nada.
3. Español claro y breve (máximo 3 párrafos cortos o viñetas).
4. Al explicar una decisión de acceso menciona las premisas (P,Q,R,S,D)
   que la provocaron según el razonamiento guardado.
```

Verificado: la pregunta *“¿Por qué TRK-201 fue enviado a inspección?”* recupera la ficha del camión y sus últimos accesos, y cita `[camiones#…]` / `[accesos#…]`. Si Ollama está caído el chat no muere: muestra los registros recuperados sin redacción (`modo: reglas`).

8. Registro de riesgos éticos

6 riesgos propios del sistema, cargados por la siembra y editables desde la GUI con matriz 5×5 inherente **y** residual:

Módulo	Riesgo	P×I	Inherente	Residual
Clasificador mixto	Fabricar categorías o datos inexistentes ante texto ambiguo	4×4	16 alto	6 medio
Motor de palabras	Subclasificación de correos con ortografía informal / jerga portuaria	4×3	12 alto	6 medio
Expediente de conductores	Datos personales de operadores (certificados, fatiga) en la base	3×5	15 alto	8 medio
Operación asistida	Dejar de contrastar decisiones automáticas por costumbre	3×4	12 alto	6 medio
Asistente contextual	Inventar respuestas plausibles sin registros suficientes	3×4	12 alto	6 medio
Verificador documental	Frenar embarques legítimos por umbrales mal ajustados	2×4	8 medio	3 bajo

Mitigaciones aplicadas en el código: pydantic estricto + reintentos + respaldo, bandera `requiere_revision_humana`, dataset con lenguaje informal, RAG con citas obligatorias, explicación visible de cada decisión y simulador para validar cambios de reglas antes de aplicarlos.

9. Pantalla de escritorio (ejercicio 2)

10 pestañas del notebook, tema oscuro `superhero`, cabecera con estado de MongoDB y del LLM, barra inferior con motor de persistencia y tamaño del dataset:

Pestaña	Qué hace
Panel	KPIs (unidades atendidas, incidentes abiertos, riesgos críticos, padrón), filtros por fecha, 3 gráficas en canvas, últimos movimientos
Acceso	Búsqueda por placa o TRK, premisas con casillas, semáforo, explicación paso a paso, bitácora filtrable
Simulador	Interruptores en vivo + tabla de verdad de A/E + tablas de Z e Y + análisis de solapamientos
Incidentes	Pegar correo → clasificar (reglas/LLM/híbrido), corregir resultado, cambiar estado, historial, filtro de revisión humana
Evaluación	Experimento sobre el dataset con barra de progreso, exactitudes, matriz de confusión, detalle por correo, histórico de corridas
Asistente	Chat RAG con fuentes citadas y sugerencias de preguntas
Riesgos	Matriz 5×5 inherente y residual + CRUD con validación pydantic
Unidades	Padrón de camiones: alta/edición/baja con validación de formato
Reportes	Vista previa + exportación CSV / JSON / PDF por colección
Configuración	Modelo Ollama, umbral de confianza, modo simulación, vigilante de turno, reconexión a MongoDB, resembrado demo

El ejercicio 1 es una ventana única: cabecera con estado de Ollama y selector de modelo, área de chat con mensajes alineados, entrada con Enter, botón **Resumen del historial** (viñetas en segunda persona generadas por el propio LLM) y reinicio de conversación.

System prompt del tutor (texto íntegro)

```
Actúa como docente universitario experto en Inteligencia Artificial.

Tu trabajo es apoyar a alumnos de licenciatura en sus dudas.

Indicaciones:

1. Responde en español y de forma clara.
2. Apóyate en ejemplos cotidianos.
3. Detalla los procedimientos uno por uno.
4. Baja el nivel técnico si el alumno va empezando.
5. Incluye fragmentos en Python cuando ayuden a entender.
6. Si el alumno se equivoca, muéstrale el error y cómo corregirlo.
7. No entregues solo la solución: expón también el porqué.
8. Termina las explicaciones largas con una idea resumen.
```

10. Cómo se comprobó

Prueba	Resultado
`python -m unittest discover -s tests`	**29/29 OK** (reglas, clasificador, esquemas)
Apertura de la app TransGuard	las 10 pestañas construyen y refrescan sin error
Apertura del tutor	ventana, chat y estado de Ollama OK
Seed en MongoDB	8 unidades, 17 accesos, 14 incidentes, 6 riesgos
Agregación categoría/semana	`$group`+`$isoWeek` con resultados por semana
Asistente RAG	cita registros reales; “sin información” cuando no hay datos
Evaluación por reglas	90.6 % categoría / 75 % prioridad
Exportación	CSV y JSON íntegros; PDF generado (fpdf2)
Modo degradado	sin MongoDB persiste en `local_fallback.json`; sin Ollama clasifica por reglas

Ejecutar las pruebas:

```
cd Ulises\ejercicio2_transguard
python -m unittest discover -s tests -v
```

11. Limitaciones conocidas

- `llama3` en CPU tarda ~90 s por clasificación; la evaluación completa en modo LLM/híbrido puede durar ~1 h sobre los 32 correos (el progreso se muestra en pantalla y puede lanzarse por separado por modo).
- El extractor de peso interpreta “toneladas/tonelada/ton/kg”; otras unidades no se capturan.
- El modo local es un respaldo de emergencia, no un reemplazo: la agregación por semana se calcula en memoria y no hay índices.
- El tutor no persiste el historial entre sesiones (vive en memoria de la ventana, a propósito, para simplificar el ejercicio).

12. Solución rápida de problemas

Síntoma	Causa probable	Acción
“LLM no disponible” en cabecera	Ollama apagado	`ollama serve` y revisar modelo en Configuración
“Respaldo local” en barra	Mongo apagado	iniciar el servicio o botón *Reintentar conexión*
Clasificación lenta	`llama3` en CPU	normal; usar modo `reglas` para pruebas rápidas
Ventana en blanco al arrancar	ttkbootstrap ausente	`pip install ttkbootstrap` (sin él funciona con ttk estándar)
Exportar PDF falla	falta fpdf2	`pip install fpdf2`